

Weight *without width.*

[Slides ↗](#) · [Measurements ↗](#) · [Paper PDF ↓](#) · [Transcript ↗](#) · [hoverBoldly ↗](#)

CONTENTS

- 01 Summary
- 02 The problem
- 03 Seventeen years of workarounds
- 04 Measuring the shift
- 05 Does it count?
- 06 Grade: the typographic answer
- 07 Why grade isn't the whole answer
- 08 Measured compensation
- 09 Does it hold?
- 10 Objections, answered
- 11 Recommendations
- 12 Open questions
- 13 Method
- 14 Sources

When text turns bold on hover, it gets wider, and everything after it moves. The web has worked around this for seventeen years. This paper measures the shift, shows that type design already solved it with grade, and describes measured compensation for the fonts that don't have it.

Disclosure: the authors make hoverBoldly and other type tools for the web, have built websites for type foundries for over fifteen years, and are building Typetin, a storefront platform for independent foundries (not yet launched). Every measurement here uses open tools (HarfBuzz, fontTools, Chromium), and no recommendation requires our library.

Summary

Bolding a label on hover, focus or the current page widens it. Across 15 popular Google Fonts variable families, going from weight 400 to 700 widened typical navigation labels by a median of 4.8% (range -0.5% to +9.4%, 105 measurements), which is 2 to 9 px per label at 16 px. In Chromium, pointer-triggered shifts are recorded as layout instability without the recent-input flag, so they are eligible to count toward Cumulative Layout Shift; the values are small.

Type design solved this long ago: the *grade* axis changes stroke weight without changing width. In Roboto Flex, moving grade across its full range changed the width of our test strings by 0.000 px. But only 1 of the 15 families ships a grade axis. For the rest, compensating letter-spacing by the measured extra width holds the line: in Chromium, with hoverBoldly applied, labels changed width by a median of 0 px (at most 0.02 px) and a pointer sweep logged no layout shifts, against a median 3.5 px change and 4–8 shifts per menu without it. The principle: **weight changes emphasis; width changes layout; an interaction state should change only the first.**

The problem

Navigation bars, tabs, filter chips and lists of links often signal hover, focus or the current page with bold. Bold glyphs are wider than regular ones, so the word grows and pushes everything after it on the line. As the pointer moves along a menu, the menu moves with it.

Seventeen years of workarounds

The problem was asked on Stack Overflow on 17 February 2009 ("Inline elements shifting when made bold on hover"). The first answer, the same day: "There is no way to avoid this." The question has been viewed about 251,000 times ([Stack Overflow](#)).

Workaround	How it works	Cost
Hidden bold copy	A pseudo-element holds a bold copy of the label so the box is pre-sized: "Pre-set the width by using an invisible pseudo-element which has the same content and styling as the parent hover style" (Stack Overflow, 2013; popularised by CSS-Tricks , 2020)	Duplicates every label; usually used for single-line labels (a grid-stacked variant handles more); keep the copy out of the accessible name with <code>visibility: hidden</code> or <code>content: attr(data-label) / ""</code>
Fake bold	<code>text-shadow</code> or <code>-webkit-text-stroke</code> thickens strokes without widening	Not real bold: "even uglier than other browser computed bold solutions" (Stack Overflow comment)
Guessed letter-spacing	A hand-tuned negative <code>letter-spacing</code> on hover	"probably font dependent" (Stack Overflow comment); breaks when the font or size changes
Fixed widths or grid columns	Give each item a fixed box	Only works when the design can afford equal boxes

None of these changes weight without changing width. They hide the width change, fake the weight, or guess.

Measuring the shift

Bold is wider in every family: median +4.8%.



Median growth of six navigation labels and one sentence, wght 400 → 700, HarfBuzz at 16 px, google/fonts at commit 9710da1, October 2026.

We shaped six navigation labels ("Home", "About", "Products", "Pricing", "Contact us", "Documentation") and a 60-character sentence with HarfBuzz at 16 px, at weight 400 and 700, in 15 Google Fonts variable families. The median growth was 4.8%; the range was -0.5% to +9.4%. Every family's median widened; one label narrowed slightly (Roboto "Home", -0.5%). "Pricing" grew 3–9% in every family. Absolute growth was about 2–9 px per label and 2.5–33 px for the sentence.

Family	Median growth	"Documentation" at 400 → 700
Work Sans	1.1%	120.1 → 121.5 px
Roboto	1.1%	109.2 → 109.8 px
Inter	4.7%	114.1 → 119.5 px
Montserrat	4.9%	127.7 → 132.0 px
Roboto Flex	7.0%	107.9 → 115.2 px
Rubik	7.7%	115.5 → 124.3 px
Open Sans	7.7%	116.1 → 125.0 px

All 105 measurements are in the [data appendix](#).

Does it count?

Google's Cumulative Layout Shift excludes shifts that happen within 500 ms of user input: "Layout shifts that occur within 500 milliseconds of user input will have the `hadRecentInput` flag set, so they can be excluded" ([web.dev](#)). But the Layout Instability explainer says: "Events caused by pointer movement or scrolling do not count as 'input' for the purpose of the recent input exclusion" ([WICG explainer](#)). Keyboard focus comes from a key press, so a focus-triggered shift is excluded; hover is not.

We tested it in Chromium 149. Hovering one bolding nav link produced a `layout-shift` entry with `hadRecentInput: false` and a value of 0.00026. Sweeping the pointer across a six-item menu produced 4–8 such entries in five of six fonts. Work Sans, whose labels grew by up to 2.6 px, produced none, which fits Chromium ignoring very small movements. So hover shifts are eligible for CLS, and the values are small: far below the 0.1 "good" threshold. The cost people notice is the visible jump.

Grade: the typographic answer

Grade adds weight. Width stays.

WEIGHT 400	Pricing	baseline
WEIGHT 700	Pricing	wider: everything after it moves
GRADE, HEAVIER	Pricing	same width (illustration)

In Roboto Flex, the GRAD axis (−200 to 150) changed measured width by 0.000 px for all seven test strings. The third row only illustrates the idea in Merriweather, which has no grade axis.

Grade is a variable-font axis that "alters only the thickness of the letterforms' strokes without changing the width of the glyph" ([Google Fonts Knowledge](#)). It was "originally conceived for printed newspapers to calibrate output from different presses". Google's documentation describes it as changing weight "without changing the layout, either horizontally, or the vertical relations of alignments", by "locking the advance widths and kerning" ([gf-docs](#)). MDN says the same: "changing the text grade doesn't change the overall layout of the text or elements around it" ([MDN](#)).

We checked. In Roboto Flex, moving **GRAD** from −200 to 150 changed the width of all seven test strings by 0.000 px, at weight 400 and at 700.

Why grade isn't the whole answer

Of the 15 families we measured, one has a grade axis: Roboto Flex. "Not all variable fonts support this specific feature, but Roboto Flex, does" ([CSS-Tricks](#), 2020). Grade is also designed for calibration, so its range may give a subtler emphasis than a full bold. Until grade is common, most web text can only get bolder by getting wider.

Measured compensation

01

Measure

Width at both weights with canvas `measureText`, off screen: no reflow.

02

Divide

Extra width ÷ number of characters.

03

Tighten

Letter-spacing reduced by that amount.

04

Bolden

Weight increased at the same moment.

`hoverBoldly`'s compensation (`src/core/adjust.ts`): an approximation of grade for fonts that only have a weight axis.

For fonts without grade, the extra width can be measured and removed. `hoverBoldly` measures the text at both weights with canvas `measureText`, off the page, so the layout isn't changed in order to measure. It divides the extra width by the number of characters and applies that as negative letter-spacing at the same moment the weight increases. It works on a whole element, word by word, or as a field that follows the cursor.

This is an approximation of grade, not grade: the letterforms are the bold letterforms, spaced tighter, and the tightening is spread evenly although the extra width comes mostly from round and wide letters.

Does it hold?

We measured the shipped library (element mode) in Chromium 149 on six of the families above (Roboto Flex, Inter, Open Sans, Rubik, Work Sans, Montserrat), six labels each.

	Width change on bold (36 labels)	Layout shifts in a pointer sweep
Plain font-weight change	median 3.5 px, max 8.9 px	4–8 per menu in 5 of 6 fonts
<code>hoverBoldly</code>	median 0 px, max 0.02 px	0 in all 6

The letter-spacing it applied ranged from -0.0004 em to -0.048 em (median -0.031 em) at 16 px.

Limits.

- It needs a variable font with a weight axis; a static bold face cannot tighten while it bolds.
- Keyboard focus is handled in element mode only; word and proximity modes respond to pointer and touch. Keep a visible focus indicator either way (WCAG 2.4.7); bold should add to it, not replace it.
- Non-zero letter-spacing turns off optional ligatures.
- Measure after web fonts load. The React hook re-applies after `document.fonts.ready`; with the plain function, call it after fonts load.
- How compensation interacts with user text-spacing overrides (WCAG 1.4.12) is untested.
- The cursor-proximity mode fixes line breaks while it runs.
- `prefers-reduced-motion` removes the transition; the weight still changes.

Objections, answered

Objection	Answer
"Use a colour change instead"	Often the right call. When a design uses weight for emphasis, keep the width.
"The pseudo-element trick works"	For single-line labels, yes, at the cost of duplicated content and accessibility care.
"It's only a few pixels"	2–9 px per label, on every hover, on every menu. The CLS cost is small; the visible jump is the problem.
"Grade already solves it"	It does, in 1 of the 15 families we measured.
"Why add JavaScript?"	Use grade, colour or the CSS copy trick where they fit. For a known font and size, a per-label em offset can also be computed at build time and shipped as CSS. <code>hoverBoldly</code> measures at runtime, so it follows the actual font, size and text.

Recommendations

1. **Type designers:** ship a grade axis in web families, with a range that reaches a visible emphasis.
2. **Designers:** spec hover, focus and current-page states with the rule "weight may change; width may not", and check the font for a grade axis before choosing bold over colour.
3. **Developers:** change weight, not width. Use `GRAD` where a font has it, and measured compensation where it doesn't. Prefer colour or underline where weight isn't needed, and keep a visible focus indicator.
4. **Standards:** searching the csswg-drafts issues, we found no proposal for emphasis without reflow. A way to say "change weight, keep the advance width" would make this the browser's job.

Open questions

- How do real sites score? We measured one link in a test page, not production navigation.
- How large a grade range do readers perceive as "bold"?
- How does compensation read in scripts other than Latin? We measured English strings only.
- How does tightened bold read at small sizes and in all caps? We measured widths, not perception.
- For families with a width axis, would reducing `width` compensate better than letter-spacing?

Method

Fonts. 15 variable families from [google/fonts](#) at commit 9710da1, read on 4 October 2026: Roboto, Open Sans, Inter, Montserrat, Noto Sans, Raleway, Nunito, Work Sans, Rubik, Roboto Flex, Source Sans 3, DM Sans, Manrope, Figtree and Mulish. Static families (Lato, Poppins) were excluded.

Measurement. Python 3.10, fontTools 4.63.0, uharfbuzz 0.56.1 (HarfBuzz 14.4.0). Strings shaped at 16 px with default features (kerning on); weight set to 400 and 700 explicitly, other axes at defaults, opsz pinned to 16. A fontTools instancer cross-check of unkered advances agreed within 0.4 percentage points.

CLS test. Chromium 149 via Playwright; a nav link styled `:hover { font-weight: 700 }` in Roboto Flex; a `PerformanceObserver` for `layout-shift` entries while the pointer moved onto the link.

Compensation test. The built library (`dist/index.js`, element mode, weight 400 → 700, no transition) on six-label menus in six families at 16 px in Chromium 149. Widths from `getBoundingClientRect` at rest and in the bold state; then a pointer swept across each menu in 6 px steps with a `layout-shift` observer.

Prior art. Every quoted source above was opened on 4 October 2026.

Sources

- Stack Overflow, [Inline elements shifting when made bold on hover](#), 2009
- Chris Coyier, [Bold on Hover... Without the Layout Shift](#), CSS-Tricks, 2020
- Greg Gibson, [Using CSS Custom Properties to Adjust Variable Font Weights in Dark Mode](#), CSS-Tricks, 2020
- Google Fonts Knowledge, [Grade](#) · Google Fonts docs, [Grade](#)
- MDN, [Variable fonts guide](#)
- web.dev, [Cumulative Layout Shift](#) · WICG, [Layout Instability](#)
- W3C, [Understanding SC 2.4.7 Focus Visible](#)
- [hoverBoldly](#) · [GitHub](#) (the authors' tool) · [our measurements](#)